DOCUMENTAÇÃO DA ARQUITETURA DO SISTEMA TASKMANAGER

Documento técnico parcial — A3: Garantia da Qualidade de Software

1. Identificação e objetivo

Sistema analisado: TaskManager

Repositório original: github.com/guuipauuli/taskmanager

O TaskManager é um sistema de gerenciamento de tarefas desenvolvido como uma prova de conceito (POC). O projeto possui uma aplicação frontend, responsável pela interface com o usuário, e um backend, responsável pelo processamento das operações e pela persistência dos dados.

O objetivo deste documento é descrever a organização arquitetural identificada no código-fonte e nos arquivos de configuração do projeto, apresentando seus componentes, tecnologias, responsabilidades e principais fluxos de comunicação.

A análise considera a estrutura do repositório, os arquivos de implementação e as informações disponíveis no README.

2. Visão geral da arquitetura

O projeto utiliza a abordagem de monorepo, ou seja, mantém o código do frontend, do backend e da infraestrutura em um único repositório. Essa organização foi adotada porque o sistema está sendo desenvolvido como uma POC, com os diferentes componentes evoluindo em conjunto e com a necessidade de simplificar a configuração do ambiente local.

A estrutura principal do repositório é organizada da seguinte forma:

```
taskmanager/
├── backend/       # Aplicação Spring Boot
├── frontend/      # Aplicação React
├── infra/         # Configuração do ambiente local
└── .github/       # Workflows de integração contínua
```

De maneira geral, o sistema funciona a partir da interação entre três partes principais:

- **Frontend:** apresenta a interface e permite que o usuário realize operações sobre as tarefas.
- **Backend:** disponibiliza uma API REST, executa as regras de aplicação e coordena o acesso aos dados.
- **Infraestrutura:** fornece os serviços necessários para a execução local, incluindo banco de dados e mensageria.

A comunicação entre frontend e backend ocorre por meio de requisições HTTP. O backend utiliza o PostgreSQL para persistência e o Kafka para publicação e consumo de eventos relacionados às tarefas.

3. Tecnologias e componentes

Componente	Tecnologia	Finalidade
Backend	Java 21	Linguagem utilizada na implementação do servidor
Framework backend	Spring Boot	Estruturação e execução da aplicação
Gerenciamento de dependências	Maven	Gerenciamento de bibliotecas e processo de build
API	Spring Web	Disponibilização dos endpoints REST
Persistência	Spring Data JPA	Integração com o banco de dados
Banco de dados	PostgreSQL 16	Armazenamento das tarefas
Migração de banco	Liquibase	Gerenciamento das alterações do esquema do banco
Mensageria	Kafka	Publicação e consumo de eventos
Coordenação do Kafka	Zookeeper	Serviço utilizado na infraestrutura de mensageria configurada
Frontend	React	Construção da interface de usuário
Linguagem frontend	TypeScript	Implementação tipada do frontend

Componente	Tecnologia	Finalidade
Ferramenta frontend	Vite	Desenvolvimento e build da aplicação frontend
Infraestrutura local	Docker Compose	Inicialização e integração dos serviços
Integração contínua	GitHub Actions	Execução de workflows separados para backend e frontend

O backend também utiliza Spring Actuator para disponibilizar informações de monitoramento, incluindo o endpoint de healthcheck.

4. Organização arquitetural do backend

O backend está organizado em pacotes que separam as responsabilidades da aplicação. A estrutura identificada apresenta os seguintes componentes:

```
backend/
└── src/main/java/com/taskmanager/backend/
    ├── api/
    ├── application/
    │   └── port/
    │       └── in/
    ├── domain/
    │   └── event/
    └── infrastructure/
        ├── config/
        ├── event/
        └── persistence/
```

4.1. API

O pacote api contém os componentes responsáveis por receber as requisições HTTP e disponibilizar os endpoints da aplicação. Entre os elementos identificados estão:

- TaskController: responsável pelos endpoints de gerenciamento de tarefas.
- TaskApiMapper: realiza a conversão entre objetos utilizados na API.
- DTOs, como CreateTaskRequest, UpdateTaskRequest e TaskResponse, que definem os dados recebidos e enviados pela API.
- GlobalExceptionHandler: centraliza o tratamento de exceções e a construção das respostas de erro.

O controller não concentra toda a lógica da aplicação. Ele encaminha as operações para os casos de uso disponibilizados pela camada de aplicação.

4.2. Application

O pacote application contém a lógica de aplicação e a coordenação dos casos de uso.

O componente TaskApplicationService implementa as operações relacionadas ao gerenciamento de tarefas.

Os casos de uso são definidos por interfaces no pacote application/port/in, como:

- CreateTaskUseCase
- DeleteTaskUseCase
- GetTaskUseCase
- ListTasksUseCase
- UpdateTaskUseCase

Essas interfaces representam as operações que podem ser executadas pela aplicação e estabelecem uma separação entre quem solicita uma operação e quem a executa.

4.3. Domain

O pacote domain representa os conceitos centrais do sistema relacionados às tarefas.

Entre os componentes identificados estão:

- Task: representa uma tarefa do sistema.
- TaskStatus: define os estados possíveis de uma tarefa.
- TaskNotFoundException: representa uma situação em que uma tarefa solicitada não é encontrada.
- TaskEvent e TaskEventType: representam eventos relacionados às tarefas.

O domínio concentra os elementos fundamentais do sistema, evitando que sua representação dependa diretamente dos detalhes de comunicação HTTP ou de persistência.

4.4. Infrastructure

O pacote infrastructure reúne os componentes responsáveis pela integração com tecnologias e serviços externos.

Na estrutura analisada, destacam-se:

- Configuração: componentes relacionados à configuração do Spring, Kafka, Liquibase e CORS.
- Persistência: componentes responsáveis pela integração com o PostgreSQL por meio do Spring Data JPA.
- Mensageria: componentes responsáveis pela publicação e pelo consumo de eventos utilizando Kafka.

Na persistência, o projeto utiliza adaptadores e mapeadores para realizar a conversão entre os objetos de domínio e as estruturas utilizadas pelo banco de dados.

Essa organização permite que a aplicação utilize recursos externos sem concentrar diretamente os detalhes dessas tecnologias em seus casos de uso.

5. Organização arquitetural do frontend

O frontend é desenvolvido com React e TypeScript, utilizando Vite como ferramenta de desenvolvimento e build.

Sua estrutura é organizada de maneira modular, separando responsabilidades comuns da aplicação das funcionalidades específicas de gerenciamento de tarefas.

Os principais diretórios identificados são:

- core: reúne elementos centrais da aplicação.
- shared: contém componentes e recursos compartilhados.
- layout: organiza elementos estruturais da interface.
- features: agrupa funcionalidades específicas do sistema.

A funcionalidade de gerenciamento de tarefas está organizada no módulo tasks.

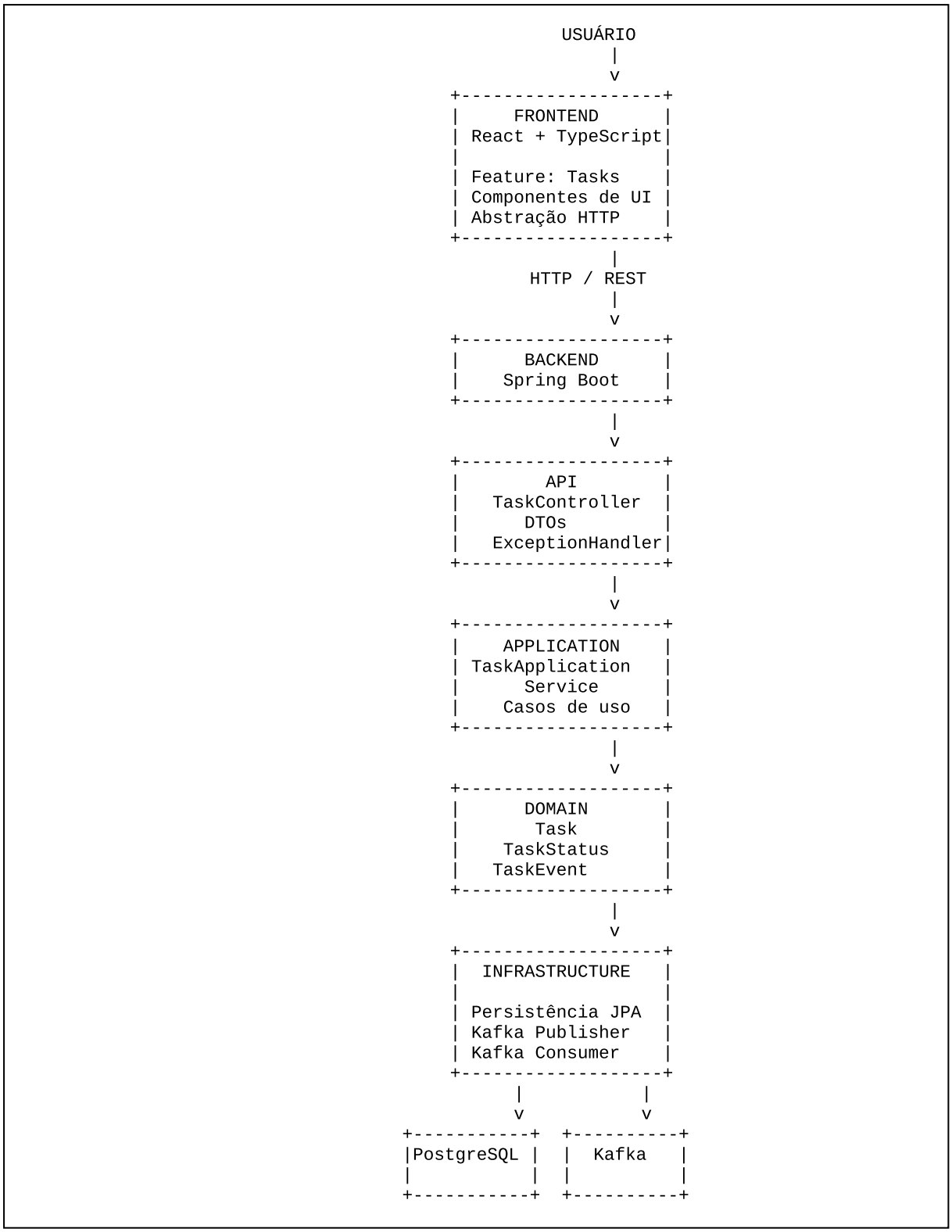
O frontend disponibiliza operações de CRUD por meio de componentes específicos, incluindo recursos reutilizáveis para formulários, modais, botões e tipografia.

Também foi identificado um hook genérico para o carregamento de dados e a execução de operações CRUD, evitando a repetição de fluxos assíncronos em diferentes componentes.

A comunicação com o backend é realizada por meio de uma abstração HTTP. O tratamento de erros de rede, timeout e indisponibilidade do backend é centralizado, enquanto os erros de validação de campos são apresentados pelos componentes de formulário.

6. Diagrama da arquitetura

O diagrama a seguir representa a organização geral dos componentes identificados no repositório.



O diagrama apresenta uma visão simplificada das responsabilidades e das integrações do sistema. O backend é organizado em componentes que separam a interface HTTP, os casos de uso, o domínio e os detalhes de infraestrutura.

A comunicação com o banco de dados ocorre por meio dos componentes de persistência. Já a integração com Kafka é realizada pelos componentes responsáveis pela publicação e pelo consumo de eventos.

7. Fluxo de funcionamento

O fluxo principal de utilização do sistema ocorre da seguinte maneira:

1. O usuário interage com a interface do frontend.
2. O frontend executa uma operação de gerenciamento de tarefas.
3. A aplicação frontend envia uma requisição HTTP para o backend.
4. O TaskController recebe a requisição e encaminha a operação para o caso de uso correspondente.
5. A camada de aplicação coordena a execução da operação.
6. Quando necessário, a aplicação utiliza os componentes de infraestrutura para acessar o PostgreSQL.
7. O resultado da operação é convertido para o formato de resposta da API.
8. O backend retorna a resposta HTTP ao frontend.
9. O frontend atualiza a interface de acordo com o resultado recebido.

Além desse fluxo, o backend possui integração com Kafka para publicação e consumo de eventos relacionados às tarefas.

8. API e operações de gerenciamento

O backend disponibiliza uma API REST para o gerenciamento das tarefas.

Os principais endpoints identificados são:

Método HTTP	**Endpoint**	**Finalidade**
GET	/api/tasks	Listar tarefas
GET	/api/tasks/{id}	Consultar uma tarefa específica
POST	/api/tasks	Criar uma tarefa
PUT	/api/tasks/{id}	Atualizar uma tarefa
DELETE	/api/tasks/{id}	Excluir uma tarefa

As operações seguem o padrão CRUD, permitindo criar, consultar, atualizar e excluir tarefas.

O sistema também possui um endpoint de healthcheck disponibilizado pelo Spring Actuator:

`/actuator/health`

Esse endpoint permite verificar o estado de saúde da aplicação.

9. Persistência e eventos

9.1. Persistência de dados

O PostgreSQL 16 é utilizado como banco de dados relacional do sistema.

O backend utiliza Spring Data JPA para realizar as operações de persistência. A estrutura do banco é gerenciada pelo Liquibase, por meio de changelogs escritos em XML.

No repositório analisado, foi identificado o changelog principal `db.changelog-master.xml` e o diretório `db/changelog/ddl/`, que contém definições da estrutura do banco, incluindo a criação da tabela de tarefas.

Observação: embora o README mencione diretórios ddl/ e dml/, o ZIP analisado apresenta o diretório ddl/. Por isso, o documento não afirma que existe um diretório dml/ separado.

9.2. Mensageria

O sistema utiliza Kafka para publicar e consumir eventos relacionados às tarefas.

Foram identificados os seguintes componentes:

- KafkaTaskEventPublisher: responsável pela publicação de eventos.
- TaskEventConsumer: responsável pelo consumo de eventos.

Os eventos representam ocorrências relacionadas às operações de tarefas. A integração utiliza Kafka como mecanismo de comunicação entre os componentes envolvidos.

O consumidor identificado registra o conteúdo recebido no log. Portanto, a análise não atribui a ele regras de negócio adicionais que não estejam demonstradas no código.

A infraestrutura local também inclui Kafka e Zookeeper, configurados por meio do Docker Compose.

10. Considerações para a Garantia da Qualidade

Separação de responsabilidades: A divisão entre API, aplicação, domínio e infraestrutura permite identificar responsabilidades específicas e facilita a análise isolada dos componentes.

Testabilidade: A existência de interfaces para os casos de uso permite verificar as operações de aplicação de maneira separada dos componentes responsáveis pela comunicação HTTP. A estrutura modular do frontend também permite analisar componentes e funcionalidades de forma mais específica.

Validação de dados: O backend possui validações para os dados recebidos nas operações de criação e atualização de tarefas. O tratamento de erros é centralizado por meio do GlobalExceptionHandler, que utiliza respostas estruturadas com ProblemDetail e informações adicionais, incluindo erros específicos de campos. Esse comportamento permite avaliar tanto a validação dos dados quanto a consistência das respostas da API.

Integração entre componentes: A comunicação entre frontend e backend, a persistência no PostgreSQL e a integração com Kafka representam pontos importantes para a análise de qualidade. Essas integrações podem ser consideradas em testes funcionais, de integração e de comportamento diante de falhas.

Infraestrutura e execução: O uso do Docker Compose permite inicializar o ambiente local com os serviços necessários para a aplicação. Os workflows de GitHub Actions separados para backend e frontend também representam mecanismos de integração contínua que podem ser considerados na análise do processo de desenvolvimento.

11. Análise e limitações

A arquitetura identificada no TaskManager apresenta uma organização modular, com separação entre as responsabilidades de interface, aplicação, domínio e infraestrutura.

O backend utiliza interfaces de casos de uso e componentes de infraestrutura para realizar operações de persistência e mensageria. O frontend possui uma estrutura modular, com funcionalidades organizadas por recursos e componentes reutilizáveis.

A utilização de Docker Compose facilita a configuração do ambiente local, enquanto os workflows de GitHub Actions permitem organizar a integração contínua dos componentes.

A análise foi baseada na estrutura do repositório, no código-fonte e nos arquivos de configuração disponibilizados.

O README também apresenta resultados de validação ponta a ponta por meio de chamadas HTTP, incluindo operações CRUD e validação de dados. Esses resultados são informações documentadas no README e não devem ser interpretados como uma execução independente realizada durante a elaboração deste documento.

Por fim, o diagrama apresentado representa uma visão arquitetural simplificada. Ele não pretende representar todos os detalhes de implementação, configurações internas ou relações entre classes do sistema.